Apuntes de JavaScript

1. Métodos de array

filter()

Devuelve un nuevo array solo con los elementos que cumplen una condición. El array original no se modifica.

```
const resultado = array.filter(elemento => condición);
```

Ejemplo:

```
const numeros = [1, 8, 3, 12, 5];

const grandes = numeros.filter(n => n > 5);
// → [8, 12]
```

map()

Devuelve un nuevo array aplicando una transformación a cada elemento. Siempre devuelve el mismo número de elementos que el original.

```
const resultado = array.map(elemento => transformación);
```

Ejemplo:

```
const nombres = ['ana', 'luis', 'eva'];

const mayusculas = nombres.map(n => n.toUpperCase());
// → ['ANA', 'LUIS', 'EVA']
```

Atención: Si usas llaves {} en el callback, map() espera un return. Sin él, devuelve un array lleno de undefined.

```
// ❌ Devuelve [undefined, undefined, undefined]:
const mal  = nombres.map(n => { n.toUpperCase() });

// ✅ Sin llaves (return implícito):
const bien = nombres.map(n => n.toUpperCase());

// ✅ Con llaves y return explícito:
const bien2 = nombres.map(n => { return n.toUpperCase(); });
```

reduce()

Recorre el array acumulando un único resultado. Puede ser un número, un string, un objeto o un array.

```
const resultado = array.reduce((acumulador, elemento) => {
  // ...
  return nuevoAcumulador;
}, valorInicial);
```

Ejemplo 1 — Sumar números:

```
const precios = [10, 25, 8, 42];

const total = precios.reduce((acc, precio) => acc + precio, 0);
// → 85
```

Ejemplo 2 — Contar ocurrencias por categoría:

```
const frutas = ['manzana', 'pera', 'manzana', 'naranja', 'pera', 'manzana'];

const conteo = frutas.reduce((acc, fruta) => {
  acc[fruta] = (acc[fruta] || 0) + 1;
  return acc;
}, {});

// → { manzana: 3, pera: 2, naranja: 1 }
```

Ejemplo 3 — Agrupar por propiedad:

```
const personas = [
  { nombre: 'Ana',  ciudad: 'Madrid'   },
  { nombre: 'Luis', ciudad: 'Valencia' },
  { nombre: 'Eva',  ciudad: 'Madrid'   },
];

const porCiudad = personas.reduce((acc, p) => {
  if (!acc[p.ciudad]) acc[p.ciudad] = [];
  acc[p.ciudad].push(p.nombre);
  return acc;
}, {});

// → { Madrid: ['Ana', 'Eva'], Valencia: ['Luis'] }
```

Atención: Incluye siempre el valor inicial (0, {}, []). Con un array vacío y sin valor inicial, reduce() lanza un TypeError.

find()

Devuelve el primer elemento que cumple la condición. Si ninguno la cumple, devuelve undefined.

```
const encontrado = array.find(elemento => condición);
```

```
const productos = [
  { id: 1, nombre: 'Teclado' },
  { id: 2, nombre: 'Ratón'   },
  { id: 3, nombre: 'Monitor' },
];
```

```
const prod = productos.find(p => p.id === 2);
// → { id: 2, nombre: 'Ratón' }
```

some() y every()

some(): devuelve true si AL MENOS UN elemento cumple la condición.

every(): devuelve true si TODOS los elementos cumplen la condición.

```
const numeros = [2, 4, 6, 7];

numeros.some(n => n % 2 !== 0);   // → true  (el 7 es impar)
numeros.every(n => n % 2 === 0);  // → false (el 7 no es par)
```

sort()

Ordena el array en su lugar (modifica el original). Necesita una función comparadora para ordenar correctamente.

```
// Números de menor a mayor:
array.sort((a, b) => a - b);

// Números de mayor a menor:
array.sort((a, b) => b - a);

// Strings en orden alfabético:
array.sort((a, b) => a.localeCompare(b));

// Por propiedad de objeto (de mayor a menor):
array.sort((a, b) => b.puntos - a.puntos);
```

Atención: sort() sin función comparadora ordena como texto: [10, 2, 1].sort() → [1, 10, 2].

2. Encadenamiento de métodos

Cuando un método de array devuelve un array, puedes llamar a otro método directamente sobre el resultado. Se puede encadenar tantas veces como sea necesario.

```
const resultado = array
  .filter(condición)    // → array filtrado
  .map(transformación)  // → array transformado
  .sort((a, b) => ...); // → array ordenado
```

Ejemplo:

```
const notas = [4, 8, 5, 9, 3, 7];

const aprobadosOrdenados = notas
  .filter(n => n >= 5)
  .sort((a, b) => b - a);
// → [9, 8, 7, 5]
```

3. const, let y var

	const	let	var
Ámbito (scope)	Bloque {}	Bloque {}	Función
Se puede reasignar	No	Sí	Sí
¿Cuándo usarlo?	Siempre que puedas	Si necesitas reasignar	Evitar

Atención: El problema clásico de var en bucles - todas las funciones capturan la misma variable

4. Funciones flecha

Son una forma más concisa de escribir funciones. Muy usadas como callbacks en map(), filter(), reduce()…

```
// Función clásica:
function sumar(a, b) { return a + b; }

// Función flecha equivalente:
const sumar = (a, b) => a + b;

// Un solo parámetro → paréntesis opcionales:
const doble = n => n * 2;

// Sin parámetros → paréntesis obligatorios:
const saludar = () => "Hola";

// Varias líneas → llaves + return:
const procesar = n => {
  const triple = n * 3;
  return triple + 1;
};
```

5. Template literals

Cadenas con comillas invertidas (`) que permiten incrustar variables o expresiones directamente con ${}.

```
const nombre = 'Ana';
const edad   = 25;

// Sin template literals:
'Hola, ' + nombre + '. Tienes ' + edad + ' años.'

// Con template literals:
`Hola, ${nombre}. Tienes ${edad} años.`

// Dentro de ${} puede ir cualquier expresión:
`El doble de ${edad} es ${edad * 2}.`
// → "El doble de 25 es 50."
```

6. Operador ternario

Un if/else compacto en una sola línea. Devuelve uno de dos valores según una condición.

```
condición ? valorSiTrue : valorSiFalse
```

```
const nota = 6;
const resultado = nota >= 5 ? 'aprobado' : 'suspenso';
// → 'aprobado'

// Útil dentro de map() para campos calculados:
const alumnos = [
  { nombre: 'Ana', nota: 7 },
  { nombre: 'Luis', nota: 4 },
];

const conEstado = alumnos.map(a => ({
  nombre:  a.nombre,
  estado:  a.nota >= 5 ? 'aprobado' : 'suspenso',
}));
```

7. Inicializar con || (OR lógico)

En JavaScript, undefined, null, 0 y "" son "falsy". Esto permite inicializar valores cuando aún no existen — muy útil dentro de reduce().

```
// Si acc[clave] no existe todavía, es undefined (falsy) → usa 0:
acc[clave] = (acc[clave] || 0) + 1;

// Para agrupar en arrays:
if (!acc[clave]) acc[clave] = [];
acc[clave].push(valor);
```

Estos dos patrones combinados con reduce() son la base de casi todos los ejercicios de agrupación.

8. Closures

Un closure es una función que recuerda las variables del entorno donde fue creada, incluso después de que ese entorno haya terminado de ejecutarse.

El caso más habitual: una función que devuelve un objeto con métodos que comparten un estado privado.

```
function crearCarrito() {
  let total = 0;

  return {
    agregar(precio) { total += precio; },
    get importe()   { return total;    },
  };
}

const carrito = crearCarrito();
carrito.agregar(12.50);
carrito.agregar(8.00);
console.log(carrito.importe); // → 20.5
```

```
// total no es accesible desde fuera:
console.log(carrito.total); // → undefined

console.log(c.valor);  // → 2
c.reiniciar();
console.log(c.valor);  // → 0

// No se puede acceder a "cuenta" desde fuera:
console.log(c.cuenta); // → undefined
```

get valor() es un getter: se accede como propiedad (`c.valor`), no como función (`c.valor()`).

Regla de oro: las variables declaradas dentro de la función externa son privadas. Solo los métodos del objeto devuelto pueden leerlas o modificarlas.

9. Formato de números

```
// Redondear a N decimales (devuelve un string):
(3.14159).toFixed(2)             // → '3.14'
(1.5).toFixed(2)                 // → '1.50'

// Convertir ese string a número de nuevo:
parseFloat((3.14159).toFixed(2)) // → 3.14

// Formato de miles con punto (convención española):
(1234567).toLocaleString('es-ES') // → '1.234.567'

// Parte entera de una división:
Math.floor(354 / 60)             // → 5  (minutos)
354 % 60                         // → 54 (segundos sobrantes)
```

parseInt(x) también quita los decimales (parseInt(5.9) → 5), pero Math.floor es más preciso con números negativos.